

Assignment 10 - Entanglement Switch

Quantum Computing and Quantum Internet (Academic Year 2025–2026)

1. Problem statement

Consider the quantum network shown in Figure 1. The network consists of k *quantum user* nodes, denoted $U_1, \dots, U_k$, arranged in a *star topology* around a central node called the *entanglement switch*.

The network supports only the generation of entangled Bell pairs between the switch and each user node, over the corresponding *link*. The user nodes are assumed to have limited quantum hardware: each can store only one qubit in quantum memory and perform single-qubit operations. The switch is a more capable quantum processor: it can store k qubits simultaneously, one memory slot per link, and apply two-qubit operations, including *Bell-state measurements* (BSMs) on the stored qubits.

The objective is to establish *end-to-end entanglement* between pairs of user nodes by means of the *entanglement swapping* protocol executed at the switch.

The users are assumed to be indistinguishable: any pair of users is an acceptable destination for an end-to-end entangled pair. The switch operates according to an *oldest-first* policy: as soon as at least two link-level Bell pairs are simultaneously stored, a BSM is performed on the two pairs that have been held in memory the longest. The measurement outcome is communicated classically to the two users involved, who apply the corresponding *Pauli correction*; the delivered end-to-end pair is then immediately consumed, the two memory slots are released, and the corresponding links resume Bell-pair generation. If more than two link-level pairs are present, the procedure is repeated until fewer than two remain.

Time is assumed to be discrete. In each time unit, every idle link performs one Bell-pair generation attempt, independently of all other links. Bell-state measurements, classical signaling, and Pauli corrections are assumed to be instantaneous.

Figure 1 illustrates:

1. the general star topology with k user nodes connected to the central switch,
2. an example configuration with $k = 4$, in which two link-level Bell pairs are stored at the switch and a BSM is about to be performed on them, and
3. the resulting end-to-end Bell pair shared by users U_1 and U_3 .

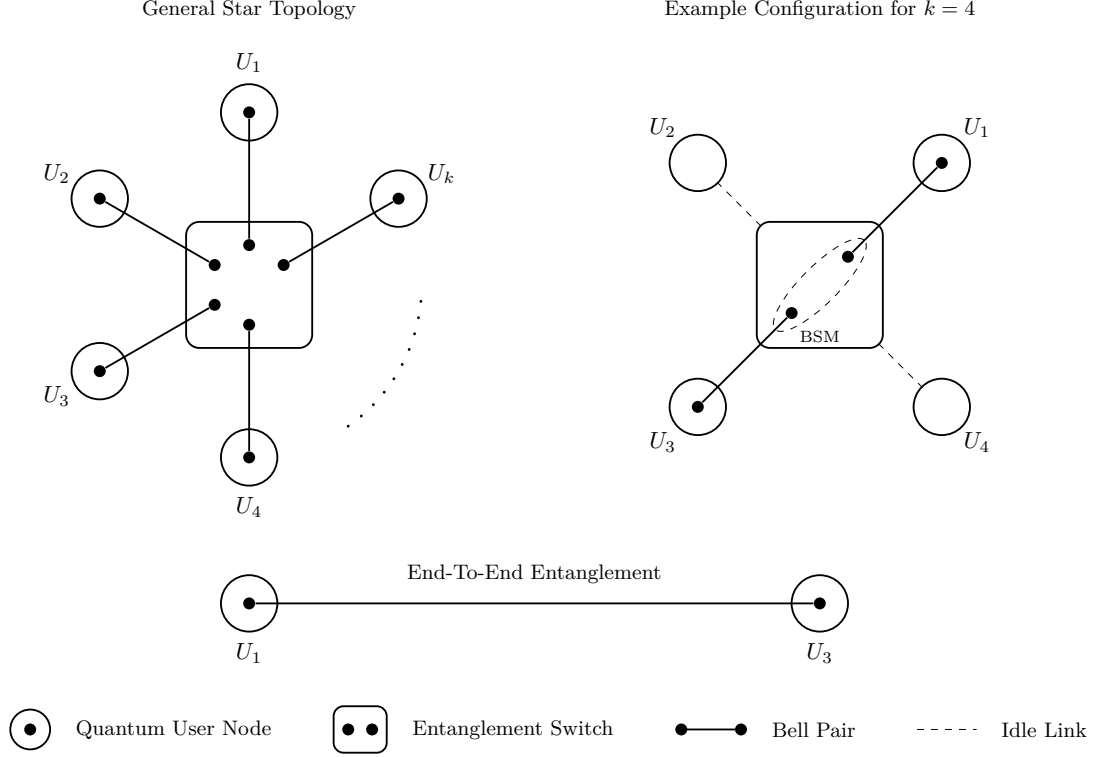


Figure 1: Star-topology quantum network with a central entanglement switch: general topology with k user nodes, an example configuration with $k = 4$ in which two link-level Bell pairs are stored at the switch and a Bell-state measurement is about to be performed, and the resulting end-to-end entangled Bell pair shared by two users.

2. Ideal benchmark

The network should first be evaluated in an *ideal setting*, in order to verify the correctness of the implementation and the expected behavior of the entanglement swapping protocol executed at the switch.

In this ideal case:

- Link-level Bell pairs are perfect states $|\Phi^+\rangle$.
- Bell-pair generation is *deterministic*: every generation attempt succeeds. Each attempt, however, still takes one discrete time unit, as prescribed by the time model of the problem statement; consequently, in every time unit, all idle links generate their link-level Bell pairs concurrently and with certainty.
- Quantum memories are noiseless.
- Bell-state measurements are ideal and, consistently with the time model, instantaneous.

In this setting, the fidelity of every delivered pair should be identically equal to 1, and the steady-state throughput should equal $\lfloor k/2 \rfloor$ end-to-end pairs per time unit.

3. Non-ideal model

After validating the ideal case, the following realistic assumptions should be introduced.

- *Werner-state link generation*: Successfully generated link-level pairs are no longer perfect Bell pairs but *Werner states* of the form

$$\rho = w |\Phi^+\rangle \langle \Phi^+| + \frac{1-w}{4} I$$

where $w \in [0, 1]$ is the *Werner parameter* and I is the identity operator on the two-qubit space. The corresponding link-level fidelity is $F_{\text{link}} = \langle \Phi^+ | \rho | \Phi^+ \rangle = (1 + 3w)/4$.

- *Probabilistic Bell-pair generation*: The preparation of link-level pairs is *probabilistic*. A single attempt to generate a pair can be modeled as a Bernoulli experiment with success probability p_{success} and failure probability $1 - p_{\text{success}}$. Each attempt takes one discrete time unit. Therefore, the number of attempts required to successfully generate a pair on a given link follows a *geometric* distribution. Attempts on distinct links are independent.
- *Memory noise*: Each quantum memory slot is affected by *depolarizing* noise with parameter p_w . For simplicity, it is acceptable to assume that all memory locations, both at the user nodes and at the switch, are subject to the same depolarizing model.

All aspects not explicitly modified by the above assumptions remain as specified in the problem statement and in the ideal benchmark; in particular, Bell-state measurements remain ideal and instantaneous.

4. Performance metrics

The following quantities should be measured and analyzed:

- *Fidelity of delivered pairs*: The fidelity of each delivered end-to-end pair should be evaluated with respect to the ideal Bell state $|\Phi^+\rangle$:

$$F = \langle \Phi^+ | \rho_{uv} | \Phi^+ \rangle$$

where ρ_{uv} is the two-qubit state held by the two users u and v after the Pauli correction determined by the BSM outcome. The fidelity should be averaged over all delivered pairs.

- *Waiting time*: The waiting time of a delivered pair is the number of time units that the older of the two swapped link-level pairs has spent in memory, from its creation to the Bell-state measurement. The waiting time should be averaged over all delivered pairs.
- *Throughput*: The throughput is the average number of end-to-end pairs delivered per time unit, evaluated in *steady state* over a sufficiently long simulation horizon.

5. Required analysis and plots

The simulation study should investigate how the above parameters affect the performance metrics, with emphasis on the following plots. When a parameter is not varied in a plot, it should be fixed at a representative value, which must be reported.

- *Throughput versus success probability*: Plot the steady-state throughput as a function of p_{success} , with separate curves representing different values of k . Three curves corresponding to distinct values of k are sufficient. This plot should highlight how the throughput increases with p_{success} , how it scales with the number of users, and how it saturates at the ideal value $\lfloor k/2 \rfloor$.
- *Fidelity versus success probability*: Plot the average fidelity of delivered pairs as a function of p_{success} , with separate curves representing different values of p_w , at a fixed representative value of w . Three curves for distinct values of p_w are sufficient. This plot should show how memory decoherence degrades the delivered fidelity and how the degradation is amplified at low values of p_{success} , when stored pairs wait longer in memory.
- *Waiting time versus number of users*: Plot the average waiting time as a function of k , with separate curves representing different values of p_{success} . Three curves for distinct values of p_{success} are sufficient. This plot should highlight how a larger number of users shortens the time that the oldest stored pair waits for a partner, and hence the decoherence experienced in memory.

6. Expected discussion

The results should include a concise interpretation of the observed trends, clearly explained and, whenever possible, supported by theoretical arguments or validated against expected behavior. In particular, the analysis should address how the performance metrics depend on the number of users k , the success probability p_{success} , the link Werner parameter w , and the memory depolarization parameter p_w . Any additional relevant effects emerging from the simulations should also be discussed.

7. Implementation requirements

The quantum-network simulation must be carried out in *Julia*, using *QuantumSavory.jl* as the quantum network simulation framework. The analysis of the simulation results and the generation of the plots may be carried out either in *Julia* or in *Python*, whichever works best.

8. Deliverables

The project deliverables are:

1. *Source code* implementing the simulation, analysis, and plot generation.
2. *Presentation slides* providing a general overview of the project, followed by a description of the adopted simulation workflow, the main results, and the final conclusions. The presentation should be designed to last no more than 15 minutes.